

EDUBRIDGE: FULL PROJECT SYNOPSIS & INTERVIEW COMPANION

Platform: Spring Boot 3.x (Java 21) REST API + ReactJS SPA + MySQL 8.x

Document Target: Technical Interview Preparation & Project Overview

1. Executive Summary & Problem Statement

In the modern tech sector, there exists a persistent mismatch between the skills taught in university classrooms and the competencies demanded by companies. Traditional educational programs are slow to adapt to changing industry requirements, leaving graduates with "skill gaps" that delay employment.

EDUBRIDGE addresses this core issue. It is a web application designed to assess, diagnostic, train, and match candidates to career opportunities. The platform allows students to build profiles and skill sets, automatically analyzes their qualifications against job requirements, suggests dynamic learning courses to plug identified skill gaps, and matches them to active jobs or internships with scoring metrics.

The project employs a modern **stateless client-server architecture**, providing high separation of concerns between its user interface layer and data engine layers.

2. Core Modules & Multi-Role User Workflows

The system implements role-based JWT authentication for four distinct portal access profiles:

- **Student Portal:** Students manage their skill matrices, view job and internship recommendations, check match percentages, run diagnostics on mismatched positions to view missing skills and suggested courses, apply for opportunities, and register for mock interview slots.
- **Recruiter Portal:** Recruiters post job openings, specify technical requirements, browse matched students with filter options, and manage applicant statuses (PENDING, APPROVED, REJECTED) with custom feedback text.
- **Mentor Portal:** Mentors schedule availability slots, view booked coaching sessions, and enter feedback after evaluating students' performance in mock technical assessments.

- **Admin Portal:** Admins monitor system-wide statistics (active users, positions, match success rates), update course mappings, and manage users.

3. Technology Stack & Technical Rationale

Backend: Java 21 & Spring Boot 3.3.x

- **Java 21 (LTS):** Leverages modern Java syntax features (like Switch Pattern Matching, Record classes, and Sequenced Collections) and guarantees virtual-threads readiness.
- **Spring Boot 3.3.x:** Accelerates development via auto-configurations. Includes `spring-boot-starter-web` for MVC REST controllers, `spring-boot-starter-validation` for automated payload sanity checks, and `spring-boot-starter-security` for filter-chain protections.
- **Spring Data JPA (Hibernate):** Handles database interactions, reducing boilerplate JDBC SQL scripts, ensuring safe database transaction boundaries, and handling connection pool scaling.
- **JSON Web Tokens (JWT):** Provides stateless session authentication, allowing requests to scale horizontally without session storage overhead on the server.

Frontend: ReactJS (v19) & Vite

- **ReactJS:** Utilizes component-based modularity, virtual DOM diffing, and single-direction state management to build a highly responsive Single Page Application (SPA).
- **Vite:** Serves as the bundler and build system, utilizing native ESM (ES Modules) to reduce development startup lag and build package sizes.
- **Axios Interceptors:** Dynamically injects authentication tokens into the `Authorization` request header and intercepts `401 Unauthorized` responses to cleanly redirect expired sessions back to the login portal.

Database: MySQL 8.x

- **MySQL Database:** Operates as the storage engine. Chosen for its transaction reliability (ACID compliance) and robust handling of many-to-many join mappings.

4. Database Architecture & Data Model

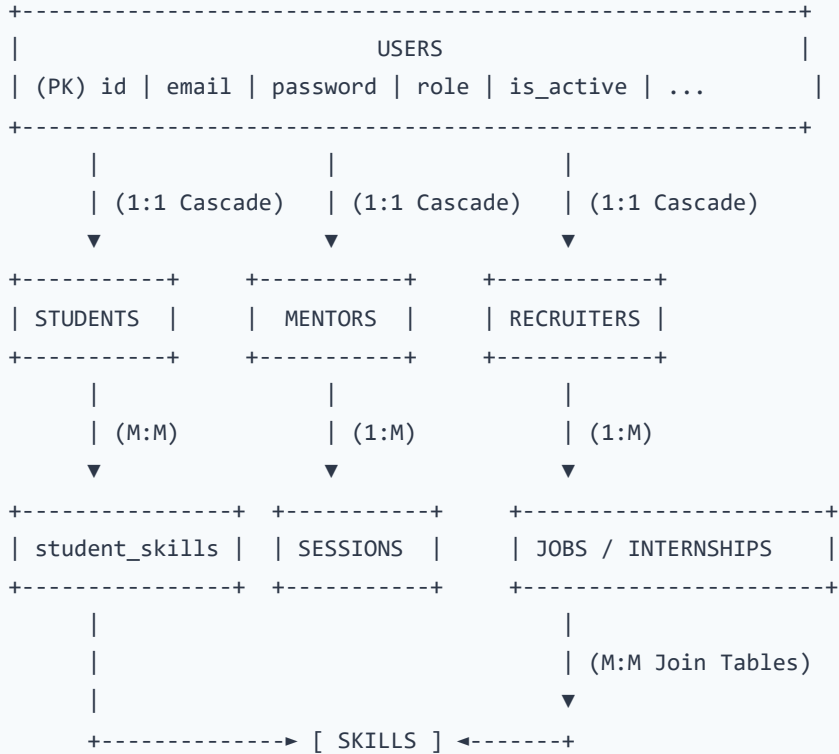
The project maps out a cleanly normalized database schema. Relationships are handled as follows:

- **User Profile Inheritance:** The `users` table contains credentials (email, hashed password, role). The `students`, `mentors`, and `recruiters` tables use their `id` column as both their Primary Key and a Foreign Key referencing `users.id` with an `ON DELETE CASCADE` constraint. This is the ****Shared Primary Key**** strategy.

- **Join Mappings:** Many-to-many associations are maintained via join tables:

- `student_skills` (associating `students` and `skills`)
- `job_skills` (associating `jobs` and `skills`)
- `internship_skills` (associating `internships` and `skills`)

Database Schema Schema-Relation Diagram (Conceptual Layout)



5. Core Business Logic & Algorithms

Algorithm 1: Dynamic Recruiter Matching Engine

The system dynamically scores jobs against student skills. The logic evaluates the intersection of skills:

1. Fetch all skills associated with Student (e.g. $S = \{\text{Java}, \text{ReactJS}\}$)
2. Fetch all jobs with their required skills from DB
3. For each Job J with required skills R (e.g. $R = \{\text{Java}, \text{Spring Boot}, \text{MySQL}\}$):
 - a. Compute intersection $\text{Inter} = S \cap R = \{\text{Java}\}$
 - b. Compute $\text{Score} = (|\text{Inter}| / |R|) * 100 = (1 / 3) * 100 = 33\%$
4. Filter out positions where $\text{Score} == 0\%$
5. Sort results in descending order of Score
6. Return sorted JobMatchDto collection to client.

Code implementation:

```
private int calculateMatchPercentage(Set<String> studentSkills, Set<Skill> requiredSkills) {
    if (requiredSkills == null || requiredSkills.isEmpty()) {
        return 100; // instant match if no requirements are specified
    }
    Set<String> reqSkillNames = requiredSkills.stream()
        .map(Skill::getName)
        .map(String::toLowerCase)
        .collect(Collectors.toSet());

    long matchCount = reqSkillNames.stream()
        .filter(studentSkills::contains)
        .count();

    return (int) (((double) matchCount / reqSkillNames.size()) * 100);
}
```

Algorithm 2: Skill-Gap Diagnostic & Course Advisor

When a student diagnostics their profile against a targeted role, the system pinpoints the differences:

1. Load student's skill set and target job's skill requirements.
2. Initialize list matchingSkills and missingSkills.
3. Iterate through each required skill:

- a. If student has it (case-insensitive), add to matchingSkills.
 - b. Else, add to missingSkills.
4. For each element in missingSkills, query DB:

SELECT * FROM courses WHERE tags LIKE %missingSkill%
5. Consolidate results, mapping matching/missing arrays and course recommendations into DTO.

Code implementation:

```
@Override
public SkillGapReportDto generateSkillGapReport(Long studentId, Long positionId, boolean
isInternship) {
    Student student = studentRepository.findById(studentId)
        .orElseThrow(() -> new ResourceNotFoundException("Student profile not found"));
    Set<String> studentSkills = student.getSkills().stream()
        .map(Skill::getName).collect(Collectors.toSet());
    Set<String> requiredSkills = getTargetSkills(positionId, isInternship);

    List<String> matching = new ArrayList<>();
    List<String> missing = new ArrayList<>();
    for (String req : requiredSkills) {
        if (studentSkills.stream().anyMatch(s -> s.equalsIgnoreCase(req))) {
            matching.add(req);
        } else {
            missing.add(req);
        }
    }

    Set<Course> recommendedCourses = new HashSet<>();
    for (String miss : missing) {
        recommendedCourses.addAll(courseRepository.findByTag(miss));
    }
    ...
}
```

6. Project Future Enhancements

If given additional time to iterate, I have planned the following product improvements:

1. **AI-Powered Career & Learning Path Optimization:** Integrate a Machine Learning service using Collaborative Filtering or content-based recommendation algorithms to match courses and paths based on successful historical student hire patterns.
2. **Automated PDF Resume Parser (NLP):** Develop an file processing service utilizing libraries like `Apache Tika` or a Python `spaCy` NLP pipeline. This will let students upload their resume, auto-extract their skill keywords, and automatically populate their skill matrices.
3. **Real-Time Notification and WebSockets:** Replace basic REST notification queries with `Spring WebSockets` & `STOMP` protocols. This will trigger instant browser notifications for students when applications change status, or when mentors post new schedule availabilities.
4. **Transition to Microservice Architecture:** Decouple the monolith application into microservices: `Auth-Service`, `Profile-Service`, `Matching-Service`, and `Mentorship-Service`. Use `Spring Cloud Gateway` for routing, `Eureka` for service registry, and `Apache Kafka` to coordinate asynchronous events.

7. Technical Interview Questions & Answers

Category A: Core Java & Spring Boot

Q1: What are the benefits of using Java 21 in this application? How does it compare to older Java 8/11 setups?

Answer: Java 21 introduces massive performance and syntax benefits. Firstly, it offers native compatibility with `Virtual Threads (Project Loom)`, which allows the server to scale to millions of concurrent requests with minimal RAM overhead compared to traditional platform threads. Secondly, syntactical additions like *Pattern Matching for Switch*, *Record Patterns*, and *Sequenced Collections* make the codebase much cleaner and reduce boilerplate.

Q2: Explain the architecture of the Security Filter Chain in Spring Boot. How does JWT authenticate requests?

Answer: When a request hits our API, Spring Security intercepts it via a filter chain. We implemented a custom `JwtAuthenticationFilter` that executes before the standard `UsernamePasswordAuthenticationFilter`. It parses the `Authorization` header, extracts the token, checks the signature using our signing key, and retrieves the username. It then queries the

`UserDetailsService` to load user roles, initializes a `UsernamePasswordAuthenticationToken`, and loads it into Spring's `SecurityContextHolder`.

Q3: What is the difference between Constructor Injection and Field Injection using `@Autowired`? Which one is used here and why?

Answer: Field injection (e.g. putting `@Autowired` on private fields) is convenient but has downsides: it makes unit testing harder because beans cannot be easily mocked without reflection, and it hides dependencies. Constructor injection is the industry standard because it enforces immutability (we can declare dependencies as `final`) and ensures that a bean cannot be instantiated in an incomplete state. In our backend, we use Constructor injection (often simplified using Lombok's `@RequiredArgsConstructor`) to enforce solid design principles.

Q4: How do you handle validation on REST payloads?

Answer: We annotate DTO arguments in our Controller methods with `@Valid`. Inside the DTO class, we use Jakarta bean validation annotations like `@NotBlank`, `@Size`, and `@Email`. If a request body fails validation, Spring throws a `MethodArgumentNotValidException`, which is intercepted by our `GlobalExceptionHandler` to return a clean, structured validation error response with a `400 Bad Request` status code.

Category B: Database, ORM & JPA

Q5: What does the setting `ddl-auto=update` do? Is it safe to use in a production environment?

Answer: The `spring.jpa.hibernate.ddl-auto=update` setting checks the existing database tables at startup and automatically modifies the database schema to match the changes made in the JPA entity classes. While it is useful during development, **it is unsafe for production**. It can result in silent data corruption or performance overhead if a field is modified or deleted. For production systems, we should disable it (`ddl-auto=none`) and use versioned database migration scripts (like **Flyway** or **Liquibase**).

Q6: Explain the N+1 select problem in JPA. How can it be solved in this codebase?

Answer: The N+1 problem occurs when fetching a list of parent entities (e.g. 1 query to get N students) results in fetching child entities individually (e.g. N additional queries to load the skills for each student). This degrades database performance. We can resolve this using **Entity Graphs** or writing JPQL queries with **Fetch Joins** (e.g. `SELECT s FROM Student s JOIN FETCH s.skills`). This instructs Hibernate to retrieve both parent and child records in a single database join query.

Q7: What is database transaction propagation? Explain the default `PROPAGATION_REQUIRED`.

Answer: Transaction propagation defines the transactional boundary behavior when one transactional method calls another. The default is `REQUIRED`. It means that if an active database transaction already exists, the called method joins it. If no transaction exists, a new one is opened. This is configured in Spring using the `@Transactional` annotation on our service layer methods.

Q8: How does `ON DELETE CASCADE` work, and where is it applied in the DB?

Answer: `ON DELETE CASCADE` is an integrity constraint configured on database foreign keys. In our schema, it connects the child profile tables (`students`, `mentors`, `recruiters`) to the parent `users` table. When an admin deletes a record from the `users` table, MySQL automatically removes the profile record with the matching `id`, preserving database integrity and avoiding orphan rows.

Category C: Frontend & State Management

Q9: What is the benefit of using React Router v7 instead of standard conditional rendering?

Answer: React Router enables client-side routing, synchronizing the page UI state with the browser URL bar. It allows page bookmarks, browser history navigation (back and forward), and nested route hierarchies with layouts. It also lets us implement Route Guards (via `ProtectedRoute.jsx`) to check authorization states before rendering a component.

Q10: Explain the purpose of Axios Interceptors in your application.

Answer: We use both request and response interceptors. The **request interceptor** automatically retrieves the logged-in user's JWT from `localStorage` and appends it to the HTTP header as `Authorization: Bearer `` for all outgoing API requests. The **response interceptor** catches errors globally; if the backend returns a `401 Unauthorized` (indicating the token has expired), the interceptor clears `localStorage` and redirects the user to `/login``.

Q11: What is Vite and why did you use it instead of Create React App (CRA)?

Answer: Create React App uses Webpack, which bundles the entire application code during development, resulting in slow start times and laggy reload times. Vite uses native browser ES Modules (ESM) to serve source files, only compilation when requested. For production, Vite uses Rollup, creating highly optimized, minified bundles with efficient tree-shaking.

Category D: Architecture & System Design

Q12: How are REST API constraints implemented in this project?

Answer: The API adheres to major REST constraints:

- **Client-Server Separation:** The frontend and backend communicate strictly via HTTP JSON payloads.
- **Statelessness:** Every HTTP request contains all context needed for authentication (JWT), eliminating server-side session dependencies.
- **Uniform Interface:** Resource paths represent collections/nouns (e.g., `/api/recruiter/jobs`) and utilize standard HTTP action verbs (GET, POST, PUT, DELETE).

Q13: How did you select the secret key for signing JWTs, and how is it protected?

Answer: The JWT secret key must be at least 256 bits for HMAC-SHA algorithms. Currently, it is stored in `application.properties` for development. For production, it must be injected as an environmental variable or managed using a secret vault (e.g. AWS Secrets Manager or HashiCorp Vault), keeping it secure from codebase commits.

Q14: Explain the difference between DTOs and Database Entities. Why not return entities directly?

Answer: Database entities mirror the schema structure and contain JPA annotations, database-specific triggers, and circular relations. Returning entities directly creates tight coupling, exposes database columns, and can trigger lazy loading serialization exceptions (or infinite loops). DTOs (Data Transfer Objects) are simple POJOs that contain only the fields required by the frontend client, ensuring clean, decoupled payloads.

Q15: How would you scale this application to support 50,000 active concurrent users?

Answer: To support 50,000 concurrent users:

1. **Horizontal Scale:** Run multiple instances of the Spring Boot application behind a load balancer (like NGINX). Because the backend is stateless (using JWT), any instance can serve any request.
2. **Caching Layer:** Implement **Redis** to cache database search results (e.g. course listings, job vacancies) to minimize direct database queries.
3. **Database Scaling:** Configure read-replicas for database query read operations and use a connection pool system (like HikariCP) configured with optimal thread sizes.



Final Interview Tip

When describing this project, focus heavily on the business algorithms (Skill Gap and Recruiter Matching Engine) and structural separation of concerns (stateless security, REST endpoints, separate client repository). Highlight your design decisions (e.g., why you used DTOs instead of entities) and explain how you would scale the system in the future.

EduBridge Interview Companion © 2026. Prepared for Career Placements.